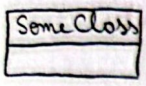
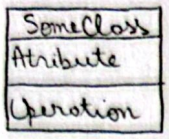
Test 4 FIS

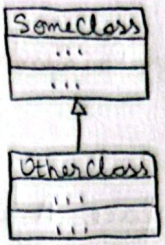
UML Class Diagrams and Sequence Diagrams



Classes

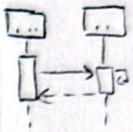


Features



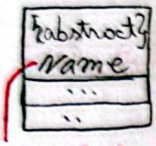
Relationships

astea sunt Diagrame de Clasă cele de secvență sunt alea



```
class SomeClass{
  ...
}
```

putem avea și metode cu italic → metode abstracte



cu italic!

```
abstract class Name {
  ...
}
```

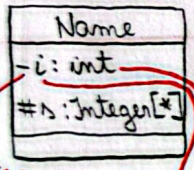



```
interface Name {
  ...
}
```

Attributes

+	public	1	exactly one
-	private	0...1	zero or at most one
#	protected	0...* or *	zero or more but NO upper limit
~	package		

Ex:



vizibilitatea

numele variabilei

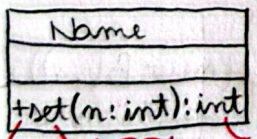
tipul ei

```
class Name {
  private int i;
  protected List<Integer> s;
}
```

mai oh cu listă pt. că dacă foloseam vector trebuia dată dimensiunea pe care n-o știm

* - oricâte elem

Operations



vizibilitatea

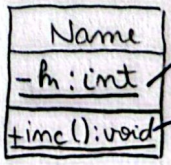
numele metodei

param. și tipul lor

tipul returnat

```
class Name {
  public int set(int n){
    ...
  }
}
```

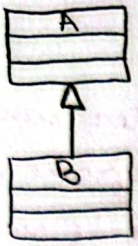
⚠ Dacă o variabilă sau o metodă e subliniată => e statică




```
private static int h;
public static void inc(){
  ...
}
```


Scanned with CamScanner

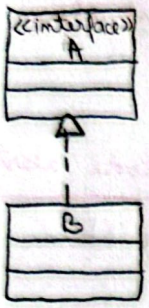
Relationships



class A {..}

class B extends A {...}

↑ extends



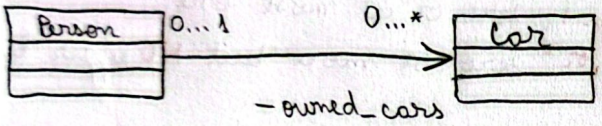
interface A {...}

class B implements A {...}

↑ implements

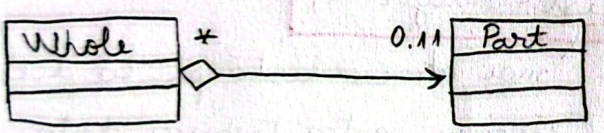
La asociere, agregare, compoziție, Car, Part, respectiv B se creează în clase
La dependență doar se folosește "temporar" fără a se instanția

Association



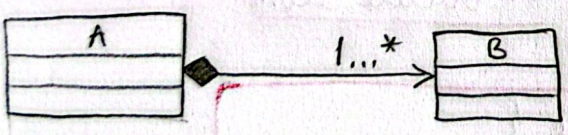

```
class PersonC
  private List<Car> owned_cars =
  new ArrayList<>();
  }
```

Aggregation



Similar cu asocierea

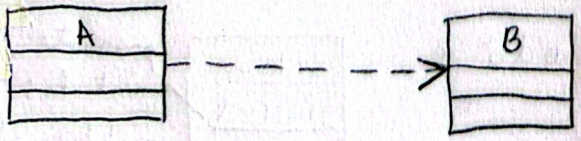
Composition



No-sharing
B objects cannot exists without their A object

```
class A {
  private List<B> my_list = new...
  public A(...) {
    my_list.add(new B(...));
  }
}
```

Dependency




```
class A {
   public void m(B x) {
      x.doS();
   }
}
```


Scanned with CamScanner

Grile

1. Afirmații false legate de Class Diagrams:

a) Pt a reprezenta o clasă abstractă sau o interfață, se pune între <<...>> - FALS, doar la interfață se pune <<...>>

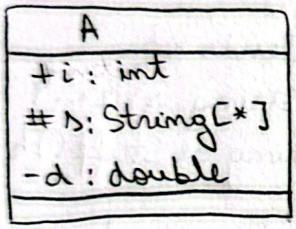
b) Liniile pt a reprezenta "o clasă extinzând clasa părinte" și "o clasă implementând o interfață sunt diferite - ADEV

la extends ↑ la implements ↑

c) Metodele statice sunt subliniate - ADEV

d) Metodele abstracte sunt subliniate - FALS, sunt scrise înclinat

2. Având diagrama clasei A, afirmațiile adevărate:



a) colecția s e protected - ADEV (#)

b) clasa A are 3 atribute: public int i, private double d și lista s care poate conține orice tip de obiecte, căci * înseamnă orice tip de obiect - FALS, conține oricâte elemente de tip String

c) [*] vine de la opțional - FALS

d) colecția s e package-private - FALS

e) clasa A are 3 atribute: public int i, private double d și o listă s fără limită - ADEV

3. Afirmații adevărate legate de Sequence Diagrams:

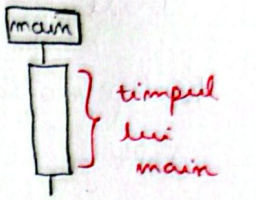
a) Diagr. de secvență oferă o privire de ansamblu asupra structurii sistemului - FALS, arată cum funcționează, nu structura

b) Diagramele-II- capturează interacțiunea dintre obiecte - ADV

c) Diagramele-II- arată relațiile dintre clase - FALS, diagramele de clase fac asta

Scanned with CamScanner

d) Diagramele^r sunt concentrate pe timp și-l reprezintă folosind axa verticală a diagramei. ADEV ex:

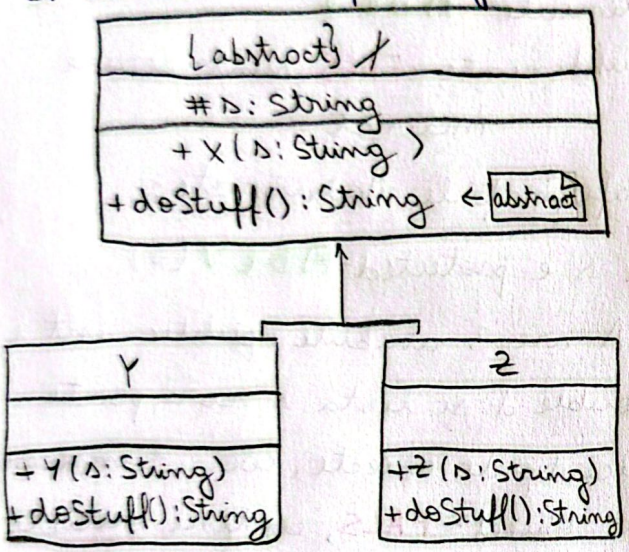


(de sus în jos curge timpul)

e) Diagramele^r sunt diagrame de interacțiune care detaliază modul în care sunt executate operațiunile - ADEV

4. Afirmații adevărate legate de Class Diagrams
tot cele de mai sus

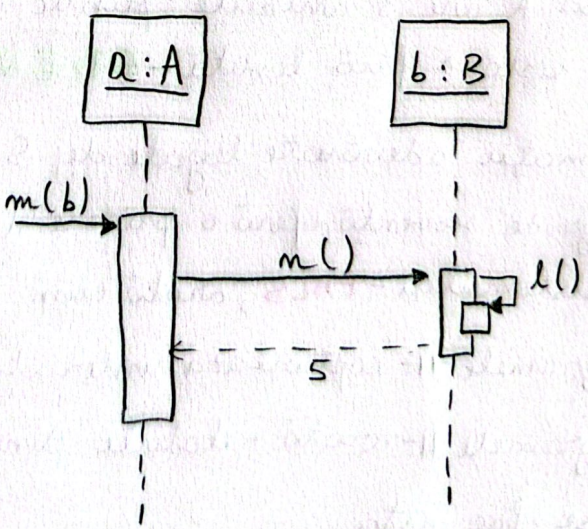
5. Codul corect pt. diagrama:




```
abstract class X{
protected String s;
public X(String s){...}
public abstract String doStuff();
}
class Y extends X{
 public Y(String s){...}
 public String doStuff(){...}
}
class Z extends X{
 public Z(String s){...}
 public String doStuff(){...}
}
```

6. Având codul, care diagramă corespunde metodei marcate cu *

```
class Main{
   A a = new A();
   B b = new B();
   a.m(b); // *
}
class A{
  public void m(B x){
      ...println(x.m());
  }
}
class B{
    public int n(){
        this.l();
        return 5;
    }
    public void l(){}
}
```



Scanned with CamScanner